Small-Scale Proxies for Large-Scale Transformer Training Instabilities

MVA 2025–2026

Tristan Beruard & Fotios Kapotos — CentraleSupélec / MVA

Introduction

Why This Matters

Large Transformer training is expensive. When a billion-scale run diverges, you lose compute, time, and signal.

Worse: failures do not reproduce at small scale. Debugging becomes guesswork.

Key Insight

High learning rates mimic large scale. Push a small model into extreme LR → same instabilities appear.

Result: failures become cheap to reproduce, study, and fix.

Paper Overview

Step 1: Identify

Two failure modes from large-scale runs:

- Attention logit growth
- Output logit divergence

Reproduced in small models at high LR.

Step 2: Mitigate

Test proposed fixes:

- **qk-layernorm**: normalize Q,K
- **z-loss**: penalty $\alpha \log^2 Z$

Step 3: Quantify

Define LR sensitivity $\mathcal{S}$:

$$\mathcal{S} = \mathbb{E}_{\eta \in [a,b]}\big[\min(\ell(A(\eta)), \ell_0) - \ell^*\big]$$

How much performance drops under bad LR.
High $\mathcal{S}$ = fragile.

Methodology

Experimental setup & training

- Decoder-only Transformer based on nanoGPT
- Architecture close to the paper: rotary embeddings, no dropout, bias-free layers
- **Next-token prediction** on OpenWebText-style dataset
- Training with **AdamW** ($\beta_1 = 0.9$, $\beta_2 = 0.95$), batch size 32, sequence length 256, bf16

Claims

Claim 1

The queries and keys in self-attention layers grow unboundedly, causing the softmax to collapse to a one-hot distribution and learning to stall. The proposed fix is *qk-layernorm*, which applies LayerNorm t queries and keys before the dot product.

Claim 2

The output logits drift to very negative values, decoupling them from the log probabilities.

Instability #1 — Attention Logit Growth

Mechanism

In self-attention, the logit between position i and j in a head of dimension d_h is:

$$z_{ij} = \frac{\langle q_i,\, k_j \rangle}{\sqrt{d_h}}$$

Both q_i and k_j grow linearly with their weight matrix norms, so z_{ij} grows **quadratically** with parameter norm — **much faster** than any other quantity in the network.

The Fix — qk-LayerNorm

Without fix

Q, **K** grow unbounded

↓

$z_{ij} \to \infty$ (quadratic growth)

↓

softmax → one-hot learning stalls

With qk-LayerNorm

Q, **K** go through LayerNorm

↓

norms pinned z_{ij} stays bounded

↓

softmax healthy training continues

What happens next

Once logits are very large, the softmax collapses to a **one-hot distribution**: every token attends to a single fixed position regardless of content. The attention mechanism becomes *content-independent* and learning stalls.

Mitigation

qk-LayerNorm: apply LayerNorm to q and k *before* computing the dot product. This pins their norms, **breaking the quadratic dependence** on parameter norm. The instability is completely prevented.

Claim 1 — Curves

Figure: Claim 1 — Placeholder for Curves

← Insert your Claim 1 figure here: train loss curves + max attention logit over training steps, with/without qk

Suggested content: training loss and max attention logit over training steps, with and without qk-layernorm.

Instability #2 — Output Logit Divergence

Mechanism

Let $y \in \mathbb{R}^V$ be the output logits, giving token probabilities:

$$p_i = \frac{e^{y_i}}{Z}, \qquad Z = \sum_j e^{y_j}$$

Without weight decay, the logits collectively drift toward **very negative values** late in training. This is distinct from attention logit growth: the divergence occurs *towards the end* of training, is *global* (all logits shift), and only manifests when **weight decay is absent**.

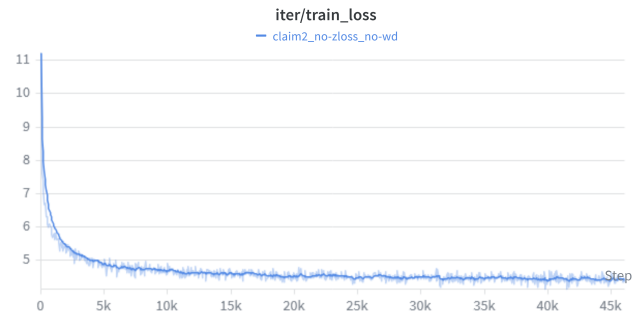
The Fix — Z-loss Regularisation

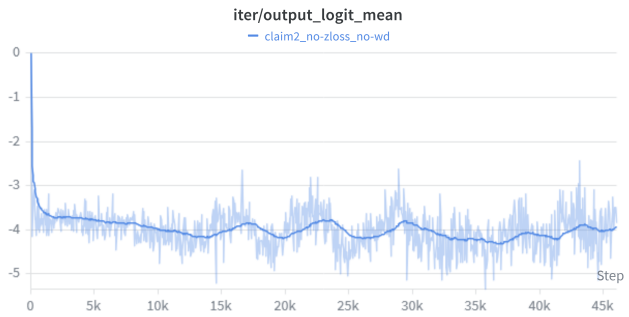
The proposed mitigation (Chowdhery et al., 2022) is an auxiliary loss term:

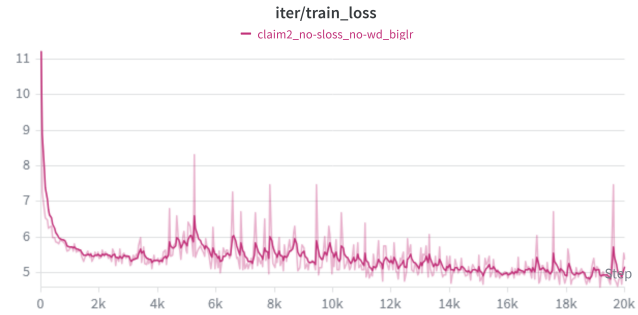
$$\mathcal{L}_{\text{z-loss}} = \alpha \log^2 Z, \qquad \alpha = 10^{-4}$$

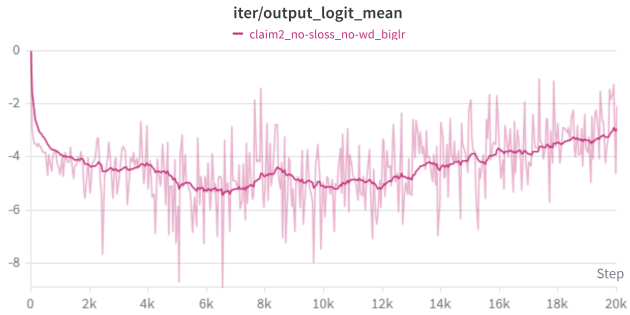
Intuition: When $Z \approx 1$ (i.e. $\log Z \approx 0$), the penalty is near zero. Any drift of Z away from 1 is *softly penalised*, keeping the logits anchored near zero and preventing collective drift to $-\infty$.

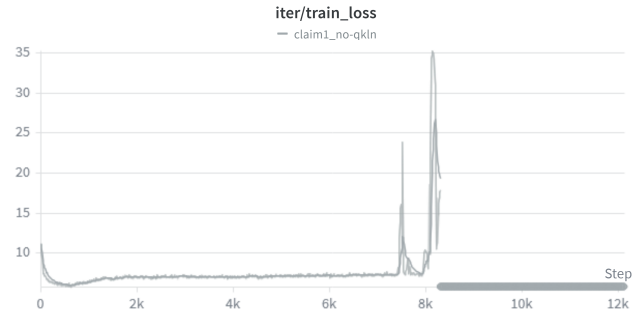
Claim 2 — Curves

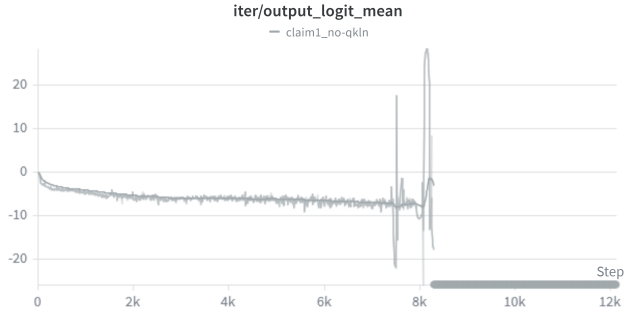












Unstable — train loss

Unstable — logit mean

Claim 2 experiments across three regimes. Top: stable training run ($lr = 0.1$). Middle: moderately stable regime with mild fluctuations (linear warmup to $lr = 0.5$). Bottom: unstable regime showing divergence. In all three cases, we did not observe a collapse of the output logit mean to $-\infty$.

LR Sensitivity — A Unified Metric for Stability

The LR Sensitivity Metric

A model is *fragile* if picking a slightly wrong learning rate leads to significantly worse performance. The paper captures this with **LR Sensitivity**:

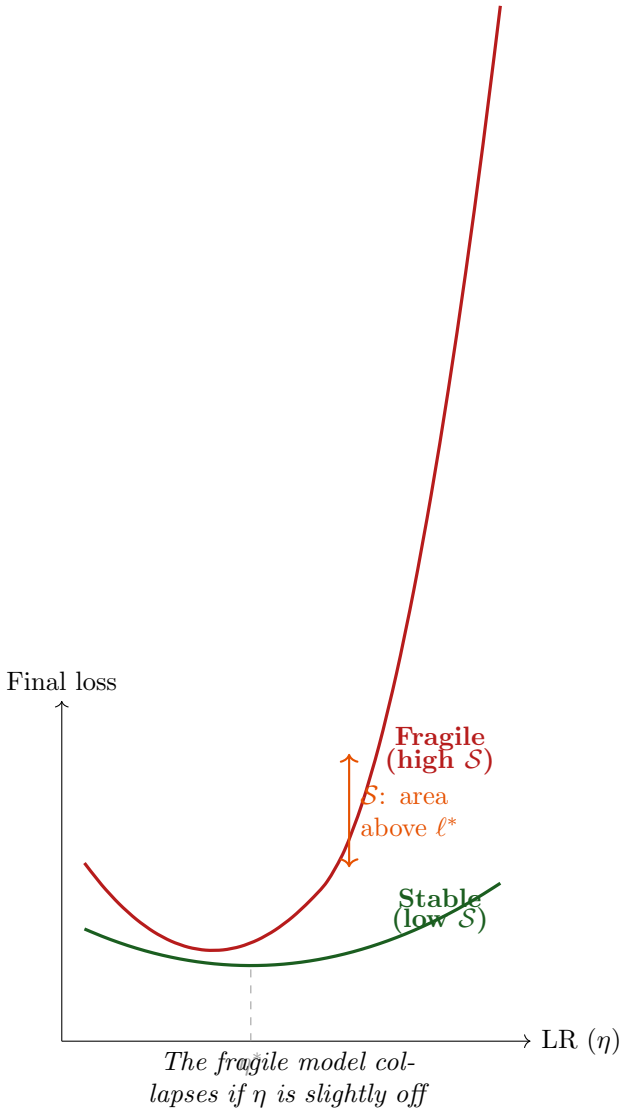
$$\mathcal{S} = \mathbb{E}_{\eta \in [a,b]}\Big[\min\big(\ell(A(\eta)),\, \ell_0\big) - \ell^*\Big]$$

- $A(\eta)$ = model trained with learning rate η
- $\ell^* = \min_\eta \ell(A(\eta))$ = best achievable loss
- ℓ_0 = loss at initialisation (caps diverged runs)
- Range $[a, b]$ spans **3 orders of magnitude**

A high $\mathcal{S}$ means the model requires very careful LR tuning and is likely to diverge if η is even slightly too large.

Key finding

Both qk-layernorm and z-loss **reduce** $\mathcal{S}$. Models with these fixes train stably across 3 orders of magnitude of LR variation. LR sensitivity still *increases with model scale*, but the mitigations slow this growth.



Beyond the Two Instabilities — Paper Insights We Highlight

Warm-up reduces $\mathcal{S}$

Longer LR warm-up substantially reduces LR sensitivity, especially at large scale. Models without warm-up cannot train stably at LR 3×10^{-1}, even with qk-layernorm.

Why it matters: warm-up gives the adaptive optimizer time to build up reliable moment estimates before committing to large gradient steps.

Depth > Width for instability

Scaling *depth* increases LR sensitivity **faster** than scaling width at the same parameter count.

However, the deepest models achieve the *lowest* final loss, suggesting a fundamental trade-off: depth buys performance but costs stability.

Predicting instability early

By tracking the *max attention logit* as a function of model scale (fitted with a quadratic), the paper **predicts** that a 4.8 B model will diverge at LR 10^{-2} before running the experiment — and the prediction is confirmed.

This is a powerful tool: identify failures *cheaply* before they occur at scale.